

Computer Organization and Architecture

Complete MCQ-Oriented Notes

IBPS SO (IT Officer) Examination - Complete Preparation Series

100% Syllabus Coverage - MCQ-Oriented - Zero Filler

Compiled by Asabhya Maanav

Table of Contents

Part A - Basic Structure of Computers	5
1.1 Von Neumann vs Harvard Architecture	5
1.2 Functional Units: CPU, Memory, I/O, System Bus	5
1.3 CPU Components: Registers, ALU, Control Unit	5
1.4 Register Types	6
1.5 Instruction Cycle: Fetch, Decode, Execute, Interrupt	6
1.6 Bus Structure: Data, Address, Control Buses	6
Part B - Machine Instructions and Addressing	8
2.1 Instruction Format and Fields	8
2.2 Instruction Length and Encoding	8
2.3 Number of Addresses: Zero, One, Two, Three-Address	8
2.4 Instruction Types	8
2.5 RISC vs CISC	9
2.6 Stack, Accumulator, Register Organizations	9
3. Addressing Modes	10
3.1 Immediate Mode	10
3.2 Direct (Absolute) Mode	10
3.3 Indirect Mode	10
3.4 Register Mode	10
3.5 Register Indirect Mode	10
3.6 Displacement / Based Mode	10
3.7 Relative (PC-Relative) Mode	10
3.8 Indexed Mode	11
3.9 Auto-Increment / Auto-Decrement	11
3.10 Effective Address Computation Summary	11
Part C - ALU, Data-Path and Control Unit	12
4.1 ALU Design and Operations	12
4.2 Data-Path Components	12
4.3 Register Transfer and Micro-operations	12
5. Control Unit	12
5.1 Hardwired Control Unit	12
5.2 Microprogrammed Control Unit	12
5.3 Micro-instructions and Micro-operations	12
5.4 Horizontal vs Vertical Microprogramming	13

5.5 Hardwired vs Microprogrammed Comparison	13
Part D - Computer Arithmetic	14
6.1 Fixed-Point Representation	14
6.2 Signed Numbers: Sign-Magnitude, 1's Complement, 2's Complement	14
6.3 Addition/Subtraction; Overflow Detection	14
6.4 Multiplication: Booth's Algorithm, Array Multiplier	14
6.5 Division: Restoring and Non-Restoring	15
6.6 Floating-Point Representation (IEEE 754)	15
Part E - Instruction Pipelining	16
7.1 Concept of Pipelining and Stages	16
7.2 Pipeline Performance: Speedup, Throughput, Efficiency	16
7.3 Ideal vs Non-Ideal Pipeline; Cycle Time	16
7.4 Pipeline Hazards	16
7.5 Computing Number of Cycles / Speedup	17
7.6 Superscalar and Out-of-Order Execution (Awareness)	17
Part F - Memory Hierarchy	19
8.1 Hierarchy Levels	19
8.2 Locality of Reference	19
8.3 Memory Types: RAM, ROM	19
8.4 Memory Organization: Word Size, Address Space	19
9. Cache Memory	21
9.1 Cache Concept: Hit, Miss, Hit Ratio	21
9.2 Mapping Techniques	21
9.3 Address Splitting and Field-Size Computation	21
9.4 Replacement Policies	22
9.5 Write Policies	22
9.6 Cache Performance: Hit Time, Miss Penalty, AMAT	22
9.7 Multi-Level Cache	23
10. Main Memory and Secondary Storage	24
10.1 Memory Interleaving	24
10.2 Memory Address Mapping / Chip Organization	24
10.3 Secondary Storage: HDD, SSD; Access Time Components	24
10.4 Virtual Memory Link	24
Part G - I/O Organization	25

11.1 I/O Modules and Device Interfacing	25
11.2 Programmed I/O (Polling)	25
11.3 Interrupt-Driven I/O	25
11.4 Direct Memory Access (DMA)	26
11.5 I/O Buses and Standards: PCI, USB	27
11.6 Synchronous vs Asynchronous Data Transfer	27

Part H - CIL MT Exam Focus and Formula Sheet 28

12.1 Effective Address Quick Table	28
12.2 Pipeline Formulas	28
12.3 Cache Field-Size and Hit-Ratio	28
12.4 AMAT	28
12.5 Hardwired vs Microprogrammed	28
12.6 Number-of-Address Reasoning	28
12.7 DMA vs Interrupt vs Programmed I/O	28
12.8 Memory Size / Address-Line Computation	28

Part A - Basic Structure of Computers

1.1 Von Neumann vs Harvard Architecture

- **Computer Organization** describes HOW hardware components are connected and operate to realize the architectural specification; **Computer Architecture** describes WHAT the machine looks like to a programmer (instruction set, data types, addressing modes).
- The **Von Neumann architecture** (also called **Princeton architecture**, 1945) uses a **single shared memory** and a **single bus** for both instructions and data.
- Because instructions and data share one bus, Von Neumann suffers the **Von Neumann bottleneck**: the CPU cannot fetch an instruction and a data word in the same cycle.
- The **Harvard architecture** uses **separate memories** and **separate buses** for instructions and data, allowing simultaneous instruction fetch and data access.
- **Modified Harvard architecture** is used in modern CPUs: a unified main memory but **split L1 caches** (separate I-cache and D-cache).

Definition

- Von Neumann: stored-program concept where program instructions and data reside in the **same read-write memory**.
- A **stored-program computer** can modify its own instructions because instructions are just data in memory.

Feature	Von Neumann	Harvard
Memory	Single shared	Separate instr/data
Buses	One bus	Two buses
Fetch + data access	Sequential	Parallel
Cost / complexity	Lower	Higher
Example	Most general CPUs	DSPs , microcontrollers, L1 cache

Memory Trick

- "Von Neumann = **V for one Vault** (one memory); Harvard = **H for two Halls** (two memories)."

CBT Trap

- **Trap**: Modern x86 CPUs are **Von Neumann at the main-memory level** but **Harvard at the L1 cache level** (modified Harvard) - examiners exploit this dual identity.
- **Trap**: The Von Neumann bottleneck is about the **bus/bandwidth**, NOT about memory size.

1.2 Functional Units: CPU, Memory, I/O, System Bus

- A computer has **5** classic functional units: **input**, **output**, **memory**, **ALU**, and **control unit**; the ALU + control unit + registers form the **CPU**.
- The **CPU (processor)** fetches, decodes and executes instructions.
- **Memory** stores instructions and data; classified as **primary (main)** and **secondary (auxiliary)**.
- **I/O units** transfer information between the computer and the external world.
- The **system bus** interconnects CPU, memory and I/O.

1.3 CPU Components: Registers, ALU, Control Unit

- The CPU contains **3** principal parts: the **register set**, the **ALU** (arithmetic-logic unit), and the **control unit (CU)**.
- The **ALU** performs arithmetic (**add**, **subtract**) and logic (**AND**, **OR**, **NOT**, **XOR**, **shift**) operations.

- The **control unit** generates **control signals** that sequence and coordinate all operations (it is the "manager").
- **Registers** are the **fastest** storage, located inside the CPU, with access time of a fraction of a **nanosecond**.

1.4 Register Types

- **PC (Program Counter)** holds the **address of the next instruction** to be fetched.
- **IR (Instruction Register)** holds the **currently executing instruction** (its opcode + operands).
- **MAR (Memory Address Register)** holds the **address** of the memory location to be accessed; its width = number of **address lines**.
- **MDR / MBR (Memory Data/Buffer Register)** holds the **data** being transferred to/from memory; its width = number of **data lines (word size)**.
- **Accumulator (AC)** is a special register that holds **intermediate ALU results**.
- **General-purpose registers (GPRs)** like R0..Rn hold operands and temporary values.
- The **Status/Flag register (PSW)** holds condition flags: **Zero (Z), Carry (C), Sign (S), Overflow (V), Parity (P)**.

Register	Holds	Width tied to
PC	Address of next instruction	Address bus
IR	Current instruction	Instruction length
MAR	Memory address	Address lines
MDR	Data word	Data lines
AC	ALU result	Word size

CBT Trap

- **Trap: MAR** width determines **addressable memory size** (2^{MAR} locations); **MDR** width determines **word size**. Do not swap them.
- **Trap: PC** points to the NEXT instruction (often already incremented during fetch), not the current one - the current one is in **IR**.

1.5 Instruction Cycle: Fetch, Decode, Execute, Interrupt

- The **instruction cycle** has phases: **fetch** -> **decode** -> **execute**, with an optional **interrupt** phase at the end.
- **Fetch micro-operations:** **MAR** <- **PC**; **MDR** <- **Memory[MAR]**, **PC** <- **PC + 1**; **IR** <- **MDR**.
- **Decode:** the control unit interprets the **opcode** to determine the operation and operands.
- **Execute:** the ALU/datapath performs the operation; results go to a register or memory.
- **Interrupt cycle:** after execute, if an interrupt is pending the CPU saves **PC** (and PSW) and loads the **ISR address**.
- The **machine cycle** is the time to fetch and execute one instruction; one instruction may take several **clock cycles**.

Memory Trick

- **"FDE-I:** Fetch, Decode, Execute, Interrupt."

CBT Trap

- **Trap: PC** <- **PC + 1** happens during **fetch**, not execute (so a branch must overwrite PC during execute).
- **Trap:** Interrupts are checked **after** the execute phase of the current instruction, not in the middle (for normal interrupts).

1.6 Bus Structure: Data, Address, Control Buses

- A **bus** is a shared set of wires carrying information between units; the **system bus** has **3** types: **data bus**, **address bus**, **control bus**.
- The **data bus** is **bidirectional**; its width = **word size** transferable per cycle.
- The **address bus** is **unidirectional** (CPU to memory/IO); n address lines address 2^n locations.
- The **control bus** carries signals like **Read, Write, Clock, Interrupt, Bus request/grant**.
- **Bus arbitration** resolves which master controls the bus: **daisy chaining**, **polling**, **independent request**.

Bus	Direction	Determines
Data	Bidirectional	Word size / throughput
Address	Unidirectional	2^n addressable cells
Control	Mixed	Timing & command

CBT Trap

- **Trap**: n address lines $\Rightarrow 2^n$ addressable locations; memory SIZE = $2^n \times \text{word_size}$.
- **Trap**: Data bus is **bidirectional**; address bus is **unidirectional** - a classic one-mark MCQ.

Part B - Machine Instructions and Addressing

2.1 Instruction Format and Fields

- A **machine instruction** is a binary code with **2** basic fields: an **opcode** (operation code) and zero or more **operand/address** fields.
- The **opcode** specifies the operation; **m** opcode bits encode up to 2^m distinct operations.
- Operand fields** specify registers, memory addresses, or immediate values; an **addressing-mode** field may select how operands are interpreted.

2.2 Instruction Length and Encoding

- Instruction length may be **fixed** (RISC, e.g., **32 bits**) or **variable** (CISC, e.g., x86 **1-15 bytes**).
- Encoding** packs opcode + mode + operands into the instruction word; trade-off is **code density vs decode simplicity**.
- Expanding opcode**: a technique where opcode length varies so that instructions with fewer operands get more opcode bits (maximizes encodings within a fixed length).

CBT Trap

- Trap**: With a fixed n-bit instruction, an **expanding opcode** lets you trade operand bits for opcode bits - count carefully in numerals.
- Trap**: More addressing modes need **more mode bits**, reducing bits available for the address field (smaller directly addressable range).

2.3 Number of Addresses: Zero, One, Two, Three-Address

- Three-address** instruction: ADD R1, R2, R3 meaning **$R1 \leftarrow R2 + R3$** ; shortest program length, longest instruction.
- Two-address**: ADD R1, R2 meaning **$R1 \leftarrow R1 + R2$** ; one operand is both source and destination.
- One-address**: ADD X meaning **$AC \leftarrow AC + X$** ; uses an implied **accumulator**.
- Zero-address**: ADD operates on the **stack** top two elements; used in **stack machines**, relies on **PUSH/POP**.

Type	Example	Meaning	Machine type
3-address	ADD A,B,C	$A=B+C$	GPR (register)
2-address	ADD A,B	$A=A+B$	GPR
1-address	ADD X	$AC=AC+X$	Accumulator
0-address	ADD	$TOS=TOS+next$	Stack

Worked Example

- To compute **$X = (A+B) \times (C+D)$** : a 3-address machine needs **3** instructions; a 0-address (stack) machine needs about **7** instructions (PUSH A, PUSH B, ADD, PUSH C, PUSH D, ADD, MUL) plus a store.

CBT Trap

- Trap**: Fewer addresses => **shorter instructions** but **longer programs** (more instructions). Three-address = shortest program.
- Trap**: Zero-address machines use **reverse Polish (postfix)** evaluation on a stack.

2.4 Instruction Types

- Instruction categories: **data transfer** (LOAD, STORE, MOVE, PUSH, POP), **arithmetic** (ADD, SUB, MUL, DIV), **logical/bit** (AND, OR, XOR, NOT, shift, rotate), **control transfer** (JMP, CALL, RET, branch), and **I/O** (IN, OUT).
- **Data transfer** moves data without changing it (no ALU op).
- **Control-transfer** instructions change the **PC** (program flow).

2.5 RISC vs CISC

- **RISC (Reduced Instruction Set Computer)**: small, **fixed-length** instruction set, **load-store** architecture, many registers, **hardwired** control, one instruction per cycle ideal.
- **CISC (Complex Instruction Set Computer)**: large, **variable-length** instructions, memory operands allowed, **microprogrammed** control, fewer registers.

Feature	RISC	CISC
Instruction length	Fixed	Variable
Instructions count	Few/simple	Many/complex
Control unit	Hardwired	Microprogrammed
Memory access	Load/Store only	Many instr access memory
Registers	Many	Fewer
Cycles/instr (CPI)	~1 (low)	>1 (high)
Pipelining	Easy	Harder
Example	ARM, MIPS, RISC-V	x86, VAX

Memory Trick

- "RISC = **R**educed = **R**egisters galore + **R**egular (fixed) format."

CBT Trap

- **Trap**: RISC has MORE registers and MORE total instructions in a program (lower code density) but **simpler** instructions - density vs simplicity.
- **Trap**: RISC favors **hardwired** control + **compiler** optimization; CISC favors **microprogrammed** control.

2.6 Stack, Accumulator, Register Organizations

- **Accumulator organization**: one implied register (AC) is source+destination => **1-address** instructions.
- **Stack organization**: operands on a **LIFO** stack => **0-address** instructions, uses **postfix** expressions.
- **General-register (GPR) organization**: multiple registers => **2 or 3-address** instructions; the dominant modern style.

CBT Trap

- **Trap**: Accumulator => 1-address, Stack => 0-address, GPR => 2/3-address. Memorize the mapping.

3. Addressing Modes

- An **addressing mode** specifies how the **effective address (EA)** of an operand is computed from the instruction fields and registers.
- The **effective address (EA)** is the actual memory address where the operand resides.
- Purposes: support **pointers, loops/arrays, relocation, indexing**, and reduce instruction size.

3.1 Immediate Mode

- The operand is **part of the instruction** itself (a constant); **EA = not applicable** (operand is in the instruction).
- **Fastest** (no memory reference for the operand); used for **constants**.
- Limitation: operand size limited by the **instruction field width**.
- Example: `MOV R1, #5` loads the constant **5**.

3.2 Direct (Absolute) Mode

- The instruction's address field holds the **effective address** directly: **EA = address field**.
- **One** memory reference to fetch the operand.
- Example: `LOAD 500` loads **M[500]**.

3.3 Indirect Mode

- The address field points to a location that **holds the effective address**: **EA = M[address field]**.
- Requires **2** memory references (one to get the pointer, one to get the operand) - **slowest memory mode**.
- Supports **pointers**.

3.4 Register Mode

- The operand is in a **register**; **EA = register (no memory access)**.
- **Fastest** operand access; example `MOV R1, R2`.

3.5 Register Indirect Mode

- The register holds the **address** of the operand in memory: **EA = contents of register**.
- **One** memory reference; used for **pointers/arrays**; example `LOAD (R1)`.

3.6 Displacement / Based Mode

- **EA = base register + constant displacement** (offset in the instruction).
- Generalizes direct, register-indirect and relative modes; used for **local variables / structures**.

3.7 Relative (PC-Relative) Mode

- **EA = PC + signed offset** (the address field is a displacement from the program counter).
- Produces **position-independent code**; used for **branches** (short forward/backward jumps).

CBT Trap

- **Trap**: In PC-relative, **EA = (updated PC, already incremented) + offset**. Many MCQs add the offset to the **post-increment** PC value.

3.8 Indexed Mode

- **EA = base address (in instruction) + index register (Xr).**
- Ideal for **array traversal**: the base is the array start, the index register steps through elements.

3.9 Auto-Increment / Auto-Decrement

- **Auto-increment**: EA = contents of register, then register is **incremented** (by operand size) - used to **walk forward** through arrays.
- **Auto-decrement**: register is **decremented first**, then used as EA - natural for **stack pop/push**.
- Increment/decrement amount = **operand size in bytes** (1, 2, 4, or 8).

3.10 Effective Address Computation Summary

Mode	EA computation	Memory refs for operand
Immediate	operand in instruction	0
Direct	EA = A	1
Indirect	EA = M[A]	2
Register	operand in R	0
Register indirect	EA = (R)	1
Displacement	EA = (R) + A	1
Relative	EA = (PC) + A	1
Indexed	EA = A + (Xr)	1
Auto-increment	EA = (R); R=R+d	1
Auto-decrement	R=R-d; EA = (R)	1

Memory Trick

- "Immediate = **value**, Direct = **address**, Indirect = **address of address**, Register = **in CPU**."

CBT Trap

- **Trap**: **Immediate** and **Register** need **0** memory references for the operand; **Indirect** needs **2**.
- **Trap**: Relative/Indexed/Based all use **addition**; the difference is WHAT is added (PC vs index reg vs base reg).
- **Trap**: Auto-increment adds the **operand width** (e.g., 4 for a 32-bit word), not always 1.

Part C - ALU, Data-Path and Control Unit

4.1 ALU Design and Operations

- The **ALU** is a combinational circuit performing **arithmetic** (add, subtract, increment) and **logic** (AND, OR, XOR, NOT, shift) operations selected by **control/function-select lines**.
- An n-bit ALU is typically built from a **ripple-carry** or **carry-lookahead** adder plus logic gates and a multiplexer to select the result.
- Status flags** produced: **Carry (C)**, **Zero (Z)**, **Sign (S)**, **Overflow (V)**.

4.2 Data-Path Components

- The **datapath** comprises **registers**, **ALU**, **internal buses**, and **multiplexers** that route data.
- Single-bus** datapath: cheap but slow (one transfer per cycle); **multi-bus** (two/three-bus) datapaths allow parallel transfers and fewer cycles per micro-operation.

4.3 Register Transfer and Micro-operations

- Register Transfer Language (RTL)** notation $R2 \leftarrow R1$ denotes copying register R1 into R2.
- A **micro-operation** is an elementary operation performed on data in registers in **one clock cycle** (e.g., load, clear, shift, add).
- Categories of micro-ops: **register transfer**, **arithmetic**, **logic**, and **shift** micro-operations.

CBT Trap

- Trap**: A micro-operation completes in **one clock pulse**; an instruction = a **sequence** of micro-operations.

5. Control Unit

- The **control unit (CU)** generates the timed **control signals** that drive the datapath to execute each instruction.
- Two implementations: **hardwired** and **microprogrammed**.

5.1 Hardwired Control Unit

- A **hardwired CU** is built from **fixed logic**: gates, decoders, a **sequence counter / state machine**, and flip-flops.
- Fast** (no microcode fetch) but **inflexible** - design changes require rewiring.
- Best for **RISC** and simple, regular instruction sets.

5.2 Microprogrammed Control Unit

- A **microprogrammed CU** stores **control words (microinstructions)** in a **control memory (control store)**, usually **ROM**.
- Each machine instruction maps to a **microprogram** (sequence of microinstructions); a **Control Address Register (CAR / microPC)** sequences them.
- Slower** (microinstruction fetch overhead) but **flexible** - changing behavior = rewriting microcode.
- Best for **CISC** / complex instruction sets.

5.3 Micro-instructions and Micro-operations

- A **microinstruction** is one word in control memory whose bits are the **control signals** for one step.

- A **micro-operation** is the hardware action a microinstruction triggers.
- **Control memory size** = number of microinstructions x microinstruction width.

5.4 Horizontal vs Vertical Microprogramming

- **Horizontal microprogramming**: **wide** microinstruction, **one bit per control signal** (unencoded), allowing **maximum parallelism**; large control store.
- **Vertical microprogramming**: **narrow** microinstruction with **encoded** fields needing a **decoder**; less parallelism but **compact**.

Feature	Horizontal	Vertical
Width	Wide	Narrow
Encoding	Unencoded (1 bit/signal)	Encoded
Parallelism	High	Low
Decoder needed	No	Yes
Control store size	Large	Small
Speed	Faster	Slower

5.5 Hardwired vs Microprogrammed Comparison

Feature	Hardwired	Microprogrammed
Implementation	Logic gates / FSM	Control memory (ROM)
Speed	Fast	Slower
Flexibility	Low (rewire)	High (rewrite microcode)
Design cost (complex ISA)	High	Low
Suited for	RISC	CISC
Modification	Difficult	Easy

Memory Trick

- "Hard**wired** = **Hardware** = **fast but rigid**; Micro**programmed** = **software in ROM** = **slow but flexible**."

CBT Trap

- **Trap**: Hardwired = **faster**, used in RISC; Microprogrammed = **flexible**, used in CISC. Examiners flip these.
- **Trap**: Horizontal = **wide + fast + parallel**; Vertical = **narrow + encoded + slow**. Don't confuse "horizontal" with "small".

Part D - Computer Arithmetic

6.1 Fixed-Point Representation

- **Fixed-point** represents numbers with an implied **binary point** at a fixed position; integers are fixed-point with the point at the **rightmost** bit.
- An n-bit unsigned number ranges **0 to $2^n - 1$** .

6.2 Signed Numbers: Sign-Magnitude, 1's Complement, 2's Complement

- **Sign-magnitude**: MSB = sign (**0=+**, **1=-**); range **$-(2^{n-1}-1)$ to $+(2^{n-1}-1)$** ; has **two zeros** (+0 and -0).
- **1's complement**: negative = bitwise **NOT** of positive; range same as sign-magnitude; also **two zeros**; needs **end-around carry** in addition.
- **2's complement**: negative = **1's complement + 1**; range **-2^{n-1} to $+(2^{n-1}-1)$** ; **single zero**; the standard representation.
- For 8 bits: 2's complement range = **-128 to +127**.

Scheme	Range (n bits)	Zeros	Negation
Sign-magnitude	$-(2^{n-1}-1) \dots (2^{n-1}-1)$	2	flip MSB
1's complement	$-(2^{n-1}-1) \dots (2^{n-1}-1)$	2	invert all bits
2's complement	-2^{n-1} $\dots (2^{n-1}-1)$	1	invert + 1

CBT Trap

- **Trap**: 2's complement has **one** extra negative number (**-2^{n-1}**) and **one** zero; sign-magnitude and 1's complement have **two** zeros.
- **Trap**: To negate in 2's complement: **invert all bits then add 1** (not just flip the sign bit).

6.3 Addition/Subtraction; Overflow Detection

- In 2's complement, **subtraction $A - B = A + (2\text{'s complement of } B)$** ; same adder hardware does both.
- **Overflow** occurs only when adding **two like-signed** numbers gives a **different-signed** result.
- Overflow detection rule: overflow = **Carry-in to MSB XOR Carry-out of MSB**.
- **No overflow** is possible when adding numbers of **opposite signs**.

CBT Trap

- **Trap**: A **carry-out** is NOT the same as **overflow**. In 2's complement a carry-out can occur without overflow; overflow uses the **Cin XOR Cout** of the sign bit.

6.4 Multiplication: Booth's Algorithm, Array Multiplier

- **Booth's algorithm** multiplies signed 2's-complement numbers by examining **pairs of bits** (current + previous) of the multiplier, handling **runs of 1s** efficiently.
- Booth recoding actions: **10 $\rightarrow$ subtract**, **01 $\rightarrow$ add**, **00/11 $\rightarrow$ do nothing** (then arithmetic shift right).
- An **array multiplier** is a combinational circuit of **AND gates + adders**; for $n \times n$ it has **$O(n^2)$** hardware and **$O(n)$** delay (carry-save reduces delay).
- Sequential **add-and-shift** multiplication takes **n** cycles for n-bit operands.

CBT Trap

- **Trap:** Booth's algorithm directly handles **signed (2's complement)** numbers and speeds up **long runs of 1s**.
- **Trap:** Array multiplier hardware grows as **$O(n^2)$** ; delay grows as **$O(n)$** .

6.5 Division: Restoring and Non-Restoring

- **Restoring division:** subtract divisor; if remainder **negative**, **restore** by adding divisor back and set quotient bit **0**; else quotient bit **1**.
- **Non-restoring division:** avoids the restore step - if remainder negative, next step **adds** instead of subtracts; faster (one add/sub per bit).
- Both take **n** iterations for n-bit operands.

Method	Restore step	Operations/bit
Restoring	Yes (when negative)	up to 2 (sub+add)
Non-restoring	No	1 (add or sub)

CBT Trap

- **Trap:** Non-restoring division does **NOT** restore; it alternates add/subtract - hence fewer operations.

6.6 Floating-Point Representation (IEEE 754)

- **IEEE 754** format: **sign** + **biased exponent** + **mantissa (significand)**.
- **Single precision (32-bit):** **1** sign + **8** exponent + **23** mantissa bits; bias = **127**.
- **Double precision (64-bit):** **1** sign + **11** exponent + **52** mantissa bits; bias = **1023**.
- **Normalized** value = $(-1)^{\text{sign}} \times 1.\text{mantissa} \times 2^{(\text{exp} - \text{bias})}$; the leading **1** is **implicit (hidden)**.
- Special values: **exp all 0s** -> **denormal/zero**; **exp all 1s** -> **infinity (mantissa 0)** or **NaN (mantissa != 0)**.

Format	Total bits	Exponent	Mantissa	Bias
Single	32	8	23	127
Double	64	11	52	1023

Memory Trick

- "Single = **1-8-23**, bias **127**; Double = **1-11-52**, bias **1023**."

CBT Trap

- **Trap:** The single-precision bias is **127** ($= 2^7 - 1$), NOT 128; double bias = **1023**.
- **Trap:** The leading **1** in the mantissa is **implicit**, so 23 stored bits give **24** bits of precision.
- **Trap:** FP addition requires **exponent alignment** (shift smaller-exponent mantissa right) before adding.

Part E - Instruction Pipelining

7.1 Concept of Pipelining and Stages

- **Pipelining** overlaps the execution of multiple instructions by splitting the datapath into **stages**, each working on a different instruction simultaneously (like an **assembly line**).
- The classic **5-stage RISC pipeline**: **IF** (instruction fetch), **ID** (instruction decode/register read), **EX** (execute/ALU), **MEM** (memory access), **WB** (write-back).
- **Pipeline registers** (latches) between stages hold intermediate results each clock.
- A **k-stage** pipeline can have up to **k** instructions in flight at once.

7.2 Pipeline Performance: Speedup, Throughput, Efficiency

- For **k** stages and **n** instructions, total cycles (ideal) = **$k + n - 1$** .
- Non-pipelined time = **$n \times k$** cycles; pipelined = **$(k + n - 1)$** cycles.
- **Speedup $S = (n \times k) / (k + n - 1)$** ; as **$n \rightarrow \text{infinity}$** , **$S \rightarrow k$** (max speedup = number of stages).
- **Throughput** = instructions completed per unit time = **$n / [(k + n - 1) \times T_{\text{clk}}]$** .
- **Efficiency** = Speedup / **$k = n / (k + n - 1)$** .

Metric	Formula
Cycles (ideal)	$k + n - 1$
Speedup	$nk / (k + n - 1)$
Max speedup	k
Efficiency	$n / (k + n - 1)$
Throughput	$n / ((k+n-1) \times T_{\text{clk}})$

7.3 Ideal vs Non-Ideal Pipeline; Cycle Time

- The **pipeline clock period** = **max stage delay + latch (register) delay**; the **slowest stage** dictates the clock.
- An **unbalanced** pipeline (one slow stage) reduces speedup; ideally stages are **balanced (equal delay)**.
- **Ideal CPI = 1**; hazards raise **CPI > 1** via stall cycles.

Worked Example

- 5-stage pipeline, 100 instructions: cycles = **$5 + 99 = 104$** ; non-pipelined = **500**; speedup = **$500/104 \approx 4.8$** .

CBT Trap

- **Trap**: Clock period is set by the **slowest** stage plus **latch delay**, not the average.
- **Trap**: Maximum speedup equals the **number of stages k** , achieved only as **$n \rightarrow \text{infinity}$** and with no hazards.

7.4 Pipeline Hazards

- A **hazard** is a condition that prevents the next instruction from executing in its designated clock cycle; **3** types: **structural, data, control**.

7.4.1 Structural Hazards

- **Structural hazard**: two instructions need the **same hardware resource** in the same cycle (e.g., single memory port for IF and MEM).
- Solutions: **separate instruction/data memories (Harvard)**, **more functional units**, or **stalling**.

7.4.2 Data Hazards (RAW, WAR, WAW)

- **Data hazard**: an instruction depends on the result of a prior instruction still in the pipeline.
- **RAW (Read After Write) = true dependency** - most common; reader needs a value not yet written.
- **WAR (Write After Read) = anti-dependency**; occurs only with **out-of-order** execution.
- **WAW (Write After Write) = output dependency**; also only with out-of-order/multiple write paths.
- Solutions: **operand forwarding (bypassing)**, **pipeline stalling/bubbles**, **register renaming** (for WAR/WAW).

Hazard	Name	When	Fix
RAW	True dependency	In-order too	Forwarding/stall
WAR	Anti-dependency	Out-of-order	Renaming
WAW	Output dependency	Out-of-order	Renaming

7.4.3 Control / Branch Hazards

- **Control (branch) hazard**: the pipeline does not know the **next instruction address** until a branch resolves, causing wrong fetches.
- **Branch penalty** = cycles wasted before the branch target is known.
- Mitigations: **stall/flush**, **branch prediction (static/dynamic)**, **delayed branch**, **branch target buffer (BTB)**.
- **Static prediction**: always taken / not-taken; **dynamic prediction**: history-based (**1-bit, 2-bit saturating counters**).

Memory Trick

- "Hazards = **S-D-C**: Structural (resource), Data (RAW/WAR/WAW), Control (branch)."

CBT Trap

- **Trap: RAW** is the **only** hazard that occurs even in **in-order** pipelines; **WAR and WAW** need out-of-order execution.
- **Trap: Forwarding** solves most RAW hazards but a **load-use** hazard still needs **1** stall cycle.
- **Trap: Register renaming** eliminates **WAR and WAW** (they are "false" dependencies), not RAW.

7.5 Computing Number of Cycles / Speedup

- With stalls, total cycles = **$k + (n - 1) + (\text{total stall cycles})$** .
- **Average CPI = $1 + (\text{stalls per instruction})$** ; speedup over non-pipelined = **$\text{CPI}_{\text{nonpipe}} / \text{CPI}_{\text{pipe}}$** (adjusted for clock).
- Branch frequency **f** with penalty **p**: **$\text{CPI} = 1 + f \times p$** .

Worked Example

- If 20% of instructions are branches with a 2-cycle penalty: **$\text{CPI} = 1 + 0.2 \times 2 = 1.4$** .

CBT Trap

- **Trap**: Add stall cycles to the base **$(k + n - 1)$** ; don't forget the **k-1** fill cycles.

7.6 Superscalar and Out-of-Order Execution (Awareness)

- **Superscalar** processors issue **multiple instructions per cycle** using multiple pipelines, achieving **$\text{CPI} < 1$** (**$\text{IPC} > 1$**).

- **Out-of-order (OoO)** execution runs ready instructions early using **register renaming** and a **reorder buffer**, exposing WAR/WAW hazards that renaming removes.
- **VLIW** packs multiple operations into one long instruction (compiler-scheduled), contrast with hardware-scheduled superscalar.

CBT Trap

- **Trap**: Superscalar's metric is **IPC > 1** (or CPI < 1); a single scalar pipeline's ideal is **CPI = 1**.

Part F - Memory Hierarchy

8.1 Hierarchy Levels

- The **memory hierarchy** (fastest/smallest/costliest at top to slowest/largest/cheapest at bottom): **registers** -> **cache (L1, L2, L3)** -> **main memory (DRAM)** -> **secondary storage (SSD/HDD)** -> **tertiary (tape)**.
- As you go down: **capacity increases**, **cost/bit decreases**, **access time increases**.
- Goal: provide the **speed of the fastest** level at the **cost of the cheapest** level using **locality**.

8.2 Locality of Reference

- Temporal locality**: a recently accessed item is likely to be accessed **again soon** (e.g., loop variables).
- Spatial locality**: items near a recently accessed address are likely to be accessed soon (e.g., array traversal); exploited by fetching **blocks/lines**.
- Sequential locality** is a special case of spatial locality.

CBT Trap

- Trap: Temporal** = same location again over **time**; **Spatial** = nearby locations in **space**. Caches exploit both; **block size** mainly exploits **spatial**.

8.3 Memory Types: RAM, ROM

- RAM (Random Access Memory)** is **volatile** read/write memory; types **SRAM** and **DRAM**.
- SRAM (Static RAM)**: uses **flip-flops (6 transistors)**; **fast**, **no refresh**, **costly**; used for **cache**.
- DRAM (Dynamic RAM)**: uses **1 transistor + 1 capacitor**; **needs periodic refresh**; **denser/cheaper/slower**; used for **main memory**.
- ROM** is **non-volatile**: **PROM** (one-time programmable), **EPROM** (UV-erasable), **EEPROM** (electrically erasable), **Flash** (block-erasable EEPROM).

Type	Cell	Refresh	Speed	Use
SRAM	6T flip-flop	No	Fast	Cache
DRAM	1T + 1C	Yes	Slower	Main memory

ROM	Erase method
PROM	Cannot erase (one-time)
EPROM	UV light
EEPROM	Electrical (byte)
Flash	Electrical (block)

CBT Trap

- Trap: SRAM** = Static = **no refresh** = cache; **DRAM** = Dynamic = **needs refresh** = main memory. "Dynamic" sounds fast but is **slower**.
- Trap: EPROM** = **UV** erase; **EEPROM** = **Electrical** erase. The extra "E" = Electrically.

8.4 Memory Organization: Word Size, Address Space

- Address space** = number of addressable units = **$2^{\text{address lines}}$** .
- Memory capacity** = **number of words x word size (bits/bytes)**.

- Number of chips to build a memory = **(required capacity) / (capacity per chip)**.

Worked Example

- A memory of **4 KB = 4096** bytes needs **12** address lines ($2^{12} = 4096$).
- To build **16K x 8** from **2K x 8** chips: need **8** chips, requiring a **3-to-8 decoder** for chip selection.

CBT Trap

- **Trap:** **1 K = $2^{10} = 1024$, 1 M = 2^{20} , 1 G = 2^{30}** when counting address lines.
- **Trap:** To extend **word size** put chips in **parallel**; to extend **number of words** stack chips and use a **decoder**.

9. Cache Memory

9.1 Cache Concept: Hit, Miss, Hit Ratio

- **Cache** is a small, fast **SRAM** between CPU and main memory holding **recently/frequently used blocks**.
- A **hit** = requested data found in cache; a **miss** = not found (fetch from lower level).
- **Hit ratio H = hits / total accesses**; **Miss ratio = $1 - H$** .
- Data moves between cache and memory in fixed-size **blocks (lines)**.

9.2 Mapping Techniques

- **Mapping** decides where a main-memory block can be placed in cache; **3** techniques: **direct**, **fully associative**, **set associative**.

9.2.1 Direct Mapping

- Each memory block maps to **exactly one** cache line: **line = (block number) mod (number of lines)**.
- Address split into **3** fields: **Tag | Line (index) | Word/Offset**.
- **Simplest, cheapest**; but high **conflict misses** (two hot blocks fight for one line).

9.2.2 Fully Associative Mapping

- A block can go into **any** cache line; address split into **2** fields: **Tag | Word/Offset**.
- Needs to compare the tag against **all** lines in parallel (**expensive CAM**); **fewest conflict misses**, **most flexible**.

9.2.3 Set Associative Mapping

- Cache divided into **sets** of **k** lines (**k-way**); a block maps to one **set** but any line within it: **set = (block) mod (number of sets)**.
- Address split into **3** fields: **Tag | Set (index) | Word/Offset**.
- A compromise: **1-way = direct**, **(all lines)-way = fully associative**.

Mapping	Address fields	Flexibility	Comparators	Conflict miss
Direct	Tag/Line/Word	Low	1	High
Set associative	Tag/Set/Word	Medium	k	Medium
Fully associative	Tag/Word	High	All	Low

9.3 Address Splitting and Field-Size Computation

- **Word/offset bits** = $\log_2(\text{block size in words/bytes})$.
- **Direct: line bits** = $\log_2(\text{number of cache lines})$; **Tag bits** = total address bits - line bits - offset bits.
- **Set associative: set bits** = $\log_2(\text{number of sets})$; number of sets = **lines / associativity**.
- **Fully associative: tag bits** = total address bits - offset bits (no index).

Worked Example

- **32-bit** address, **block size 16 bytes**, **cache 16 KB**, direct mapped: offset = **4 bits** ($16=2^4$); lines = $16\text{KB}/16\text{B}$ = **1024** -> index = **10 bits**; tag = $32 - 10 - 4$ = **18 bits**.
- Same cache **4-way set associative**: sets = $1024/4$ = **256** -> set index = **8 bits**; tag = $32 - 8 - 4$ = **20 bits**.

Memory Trick

- "Direct = **Tag-Line-Word**; Set = **Tag-Set-Word**; Full = **Tag-Word** (no index)."

CBT Trap

- **Trap**: Increasing associativity **increases tag bits** and **decreases index bits** (fewer sets).
- **Trap**: Fully associative has **no index/line field** - all non-offset bits are **tag**.
- **Trap**: Number of **comparators** = associativity (direct=1, fully=all lines).

9.4 Replacement Policies

- On a miss in a full set/cache, a **replacement policy** chooses the victim block.
- **LRU (Least Recently Used)**: evict the **least recently accessed** block; good but needs tracking.
- **FIFO (First In First Out)**: evict the **oldest loaded** block; can suffer **Belady's anomaly**.
- **Random**: evict a **random** block; cheapest hardware.
- **Optimal (OPT/MIN)**: evict the block **used farthest in the future**; theoretical lower bound (not implementable).

CBT Trap

- **Trap**: **Belady's anomaly** (more frames -> more misses) can occur with **FIFO**, never with **LRU or Optimal (stack algorithms)**.
- **Trap**: **Optimal** is unrealizable (needs future knowledge); used only as a **benchmark**.

9.5 Write Policies

- **Write-through**: write to **cache and main memory simultaneously**; memory always consistent, but **higher write traffic**.
- **Write-back (copy-back)**: write only to **cache**; mark a **dirty bit**; write to memory only on **eviction**; less traffic, risk on power loss.
- **Write-allocate**: on a write miss, **load the block** into cache first (pairs with write-back).
- **No-write-allocate**: on a write miss, write **directly to memory**, skip cache (pairs with write-through).

Policy	When written to memory	Pairs with
Write-through	Every write	No-write-allocate
Write-back	On eviction (dirty)	Write-allocate

CBT Trap

- **Trap**: **Write-back** uses a **dirty bit**; **write-through** does not need one.
- **Trap**: Typical pairing: **write-back + write-allocate**, **write-through + no-write-allocate**.

9.6 Cache Performance: Hit Time, Miss Penalty, AMAT

- **Hit time** = time to access data in cache; **Miss penalty** = extra time to fetch a block from the next level.
- **AMAT (Average Memory Access Time)** = **Hit time + Miss ratio x Miss penalty**.
- Alternatively: **AMAT** = $H \times T_c + (1 - H) \times (T_c + T_m)$ or $H \times T_c + (1-H) \times T_m$ depending on convention (simultaneous vs hierarchical access).
- **Effective access time (2-level)** = $H \times T_{\text{cache}} + (1 - H) \times T_{\text{memory}}$.
- **Multi-level AMAT** = $\text{HitTime_L1} + \text{MissRate_L1} \times (\text{HitTime_L2} + \text{MissRate_L2} \times \text{MissPenalty_L2})$.

Worked Example

- Hit time = **1 ns**, miss penalty = **100 ns**, hit ratio = **95%**: $AMAT = 1 + 0.05 \times 100 = 6 \text{ ns}$.
- 2-level: L1 hit 1ns, L1 miss rate 10%, L2 hit 10ns, L2 miss rate 5%, mem 100ns: $AMAT = 1 + 0.10 \times (10 + 0.05 \times 100) = 1 + 0.10 \times 15 = 2.5 \text{ ns}$.

Memory Trick

- " **$AMAT = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$** " - the single most-tested cache formula."

CBT Trap

- **Trap**: Watch whether the problem uses **hierarchical** ($T_c + T_m$ on miss) or **simultaneous/look-aside** (T_m only on miss) access - the AMAT formula differs.
- **Trap**: Miss rate must be a **fraction** (0.05), not a percentage (5) in the formula.

9.7 Multi-Level Cache

- **Multi-level caches** (**L1**, **L2**, **L3**) bridge the speed gap: **L1** small/fast (often split I/D), **L2/L3** larger/slower (unified, often shared).
- **Local miss rate** = misses at that level / accesses to that level; **Global miss rate** = misses at that level / total CPU accesses.
- **Inclusive** caches duplicate L1 contents in L2; **exclusive** caches do not.

CBT Trap

- **Trap**: **Global miss rate** of L2 = **L1 miss rate** x **L2 local miss rate**.

10. Main Memory and Secondary Storage

10.1 Memory Interleaving

- **Memory interleaving** spreads consecutive addresses across multiple **memory banks/modules** so they can be accessed in parallel, increasing **bandwidth**.
- **Low-order interleaving**: consecutive words in **different banks** (best for sequential access).
- **High-order interleaving**: consecutive words in the **same bank** (used for fault isolation, not bandwidth).
- With **m** banks, ideal bandwidth improves up to **mx**.

CBT Trap

- **Trap**: **Low-order** interleaving boosts **sequential bandwidth**; high-order does **not**.

10.2 Memory Address Mapping / Chip Organization

- A memory chip is organized as a **2D array** of rows x columns; addresses split into **row address** and **column address** (multiplexed in DRAM via **RAS/CAS**).
- **Address decoding**: a **decoder** selects the chip/word from high-order address bits.

10.3 Secondary Storage: HDD, SSD; Access Time Components

- **HDD access time** = **seek time** + **rotational latency** + **transfer time** (+ controller overhead).
- **Seek time**: move the head to the track; **rotational latency**: wait for the sector (average = **half a rotation**); **transfer time**: read the data.
- **Average rotational latency** = $(1/2) \times (60 / \text{RPM})$ seconds.
- **SSD** uses **NAND flash**: **no moving parts**, **much faster random access**, limited **write endurance (wear)**, uses **wear-leveling**.

Aspect	HDD	SSD
Moving parts	Yes	No
Random access	Slow (seek)	Fast
Access time	ms	microseconds
Wear	Low	Limited writes

Worked Example

- At **7200 RPM**: average rotational latency = $0.5 \times 60/7200 \text{ s} = 4.17 \text{ ms}$.

CBT Trap

- **Trap**: **Average** rotational latency = **half** a full rotation time, not a full rotation.
- **Trap**: HDD total access time = **seek** + **rotational latency** + **transfer**; don't omit rotational latency.

10.4 Virtual Memory Link

- **Virtual memory** uses main memory as a **cache for disk**, giving each process a large **logical address space**; managed by **paging** (see Operating Systems).
- The **TLB (Translation Lookaside Buffer)** caches recent **page-table** translations.

Part G - I/O Organization

11.1 I/O Modules and Device Interfacing

- An **I/O module (interface)** connects devices to the system bus and contains **data, status, and control registers**.
- **I/O addressing: memory-mapped I/O** (I/O shares the memory address space, uses normal load/store) vs **isolated/port-mapped I/O** (separate I/O address space, special **IN/OUT** instructions and an **IO/M** control line).

Scheme	Address space	Instructions
Memory-mapped	Shared with memory	Normal LOAD/STORE
Isolated (port)	Separate	IN/OUT

CBT Trap

- **Trap: Memory-mapped I/O** reduces usable memory address space but needs **no special instructions**; isolated I/O preserves memory space but needs **IN/OUT**.

11.2 Programmed I/O (Polling)

- **Programmed I/O**: the CPU **continuously polls** (busy-waits on) the device status flag and transfers each word itself.
- **Simplest** but **wastes CPU time** (busy waiting); throughput limited by CPU.

CBT Trap

- **Trap**: Programmed I/O = **CPU does everything** and **polls**; the device cannot interrupt - maximum CPU waste.

11.3 Interrupt-Driven I/O

- **Interrupt-driven I/O**: the device **interrupts** the CPU when ready, so the CPU does other work meanwhile (no busy-waiting).
- On interrupt: CPU finishes the current instruction, **saves PC/PSW**, runs the **ISR**, then **returns**.

11.3.1 Interrupt Types; Maskable vs Non-Maskable

- **Maskable interrupts** can be **disabled/ignored** by the CPU (via the interrupt-enable flag); **non-maskable (NMI)** cannot be ignored (used for **critical events** like power failure).
- **Hardware interrupts** come from devices; **software interrupts (traps)** are generated by instructions; **exceptions** arise from errors (divide-by-zero, page fault).

11.3.2 Interrupt Priority; Daisy Chaining

- When multiple interrupts occur, a **priority** scheme decides order.
- **Daisy chaining**: devices connected in **series**; the **closest device to the CPU has highest priority**; uses an **acknowledge** signal passed down the chain.
- **Parallel priority** uses a **priority encoder** and mask register.

11.3.3 ISR and Vectored Interrupts

- The **Interrupt Service Routine (ISR)** is the code that handles the interrupt.
- **Vectored interrupt**: the device supplies an **interrupt vector** (address/ID) so the CPU jumps **directly** to the correct ISR via the **interrupt vector table**.

- **Non-vectorized interrupt:** the ISR address is **fixed/predetermined**; CPU polls to find the source.

Concept	Vectorized	Non-vectorized
ISR address source	Device supplies vector	Fixed
Speed	Faster	Slower (poll)

CBT Trap

- **Trap: NMI** (e.g., power failure) **cannot be masked**; maskable interrupts can be disabled.
- **Trap:** In **daisy chaining**, priority is by **physical position** - nearest to CPU = **highest**.
- **Trap:** **Vectorized** = device gives the address; **non-vectorized** = fixed address. Don't confuse with maskable/non-maskable.

11.4 Direct Memory Access (DMA)

- **DMA** lets an **I/O device transfer data directly to/from memory** without CPU involvement per word; the CPU is freed for other work.
- The **DMA controller (DMAC)** becomes a **bus master**.

11.4.1 DMA Controller Operation

- DMA registers: **address register** (memory location), **count register** (number of words), **control register** (read/write, mode).
- Steps: CPU programs the DMAC; device requests via **DMA request (DRQ)**; DMAC requests the bus (**HOLD**); CPU grants (**HLDA**); DMAC transfers data; on completion it **interrupts** the CPU.

11.4.2 Cycle Stealing vs Burst Mode

- **Cycle stealing:** DMAC transfers **one word at a time**, "stealing" individual bus cycles between CPU accesses; CPU slows slightly but keeps running.
- **Burst (block) mode:** DMAC transfers a **whole block** while holding the bus; CPU is **stalled** during the burst; fastest transfer.
- **Transparent/hidden mode:** DMA transfers only when the CPU is **not using the bus** (no slowdown, slowest DMA).

Mode	Bus held	CPU impact	Speed
Cycle stealing	1 word at a time	Slight slowdown	Medium
Burst	Whole block	Stalled	Fastest
Transparent	Only when CPU idle	None	Slowest

11.4.3 Advantages over Programmed/Interrupt I/O

- DMA gives the **highest throughput** for **bulk transfers**: the CPU is interrupted only **once per block** (not per word).
- Programmed I/O: CPU per-word + polls; interrupt I/O: CPU per-word but no poll; DMA: CPU only sets up + final interrupt.

Method	CPU involvement	Interrupts	Best for
Programmed I/O	Every word + poll	None	Simple/slow devices
Interrupt I/O	Every word (in ISR)	Per word	Moderate rate
DMA	Setup only	Per block	Bulk/high-speed

Memory Trick

- "DMA = **Direct Memory Access** = CPU out of the loop = **one interrupt per block**."

CBT Trap

- **Trap**: In DMA the CPU is interrupted **once per block**, whereas interrupt-driven I/O interrupts **once per word/byte**.
- **Trap**: **Burst mode** stalls the CPU completely; **cycle stealing** only slows it. "Cycle stealing" steals **single** cycles.
- **Trap**: DMA does NOT eliminate the CPU entirely - the CPU still **programs the DMAC** and handles the **completion interrupt**.

11.5 I/O Buses and Standards: PCI, USB

- **PCI (Peripheral Component Interconnect)**: a **parallel** expansion bus; **PCIe (PCI Express)** is the modern **serial, point-to-point, lane-based** successor.
- **USB (Universal Serial Bus)**: a **serial, hot-pluggable, tiered-star** bus; host-controlled; supports many device classes.
- USB is **host-pollled** (devices do not initiate); supports **plug-and-play** and power delivery.

CBT Trap

- **Trap**: **PCIe** and **USB** are **serial**; legacy **PCI** is **parallel**. PCIe is **point-to-point**, not a shared bus.

11.6 Synchronous vs Asynchronous Data Transfer

- **Synchronous transfer**: sender and receiver share a **common clock**; fast but requires matched timing.
- **Asynchronous transfer**: no shared clock; uses **handshaking** (**strobe** or **handshake** signals like ready/acknowledge) and **start/stop bits**.

Type	Clock	Mechanism	Speed
Synchronous	Shared	Clock edges	Faster
Asynchronous	None	Handshake /start-stop	Slower

CBT Trap

- **Trap**: **Asynchronous** uses **handshaking/start-stop bits** (no common clock); synchronous uses a **shared clock**.

Part H - CIL MT Exam Focus and Formula Sheet

12.1 Effective Address Quick Table

Mode	EA
Immediate	operand in instr
Direct	$EA = A$
Indirect	$EA = M[A]$
Register	operand in R
Register indirect	$EA = (R)$
Displacement	$EA = (R) + A$
Relative	$EA = (PC) + A$
Indexed	$EA = (Xr) + A$

12.2 Pipeline Formulas

- Cycles = $k + n - 1$; Speedup = $nk/(k+n-1)$; max = k ; Efficiency = $n/(k+n-1)$; Throughput = $n/((k+n-1)T_{clk})$.
- With branches: CPI = $1 + f \times \text{penalty}$.

12.3 Cache Field-Size and Hit-Ratio

- Offset bits = $\log_2(\text{block size})$; index/line bits = $\log_2(\# \text{lines or \#sets})$; tag = remaining bits.
- Hit ratio $H = \text{hits}/\text{total}$; miss = $1 - H$.

12.4 AMAT

- $AMAT = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$.
- Effective access (2-level) = $H \times T_c + (1-H) \times T_m$.

12.5 Hardwired vs Microprogrammed

- Hardwired = fast, rigid, RISC; Microprogrammed = slow, flexible, CISC.

12.6 Number-of-Address Reasoning

- 3-address = shortest program; 0-address = stack/postfix; 1-address = accumulator.

12.7 DMA vs Interrupt vs Programmed I/O

- Programmed = CPU per word + poll; Interrupt = CPU per word, no poll; DMA = setup + 1 interrupt/block.

12.8 Memory Size / Address-Line Computation

- n address lines $\Rightarrow 2^n$ locations; capacity = $2^n \times \text{word size}$; chips = total/per-chip.
- $1K=2^{10}$, $1M=2^{20}$, $1G=2^{30}$.

Final Memory Trick

- "AMAT, $k+n-1$, Tag-Index-Offset, FDE, RAW/WAR/WAW - the five pillars of COA MCQs."